

Guía de ejercicios - JEE y Java Server Pages (III)



¡Hola! Te damos la bienvenida a esta nueva guía de estudio.

¿En qué consiste esta guía?

La siguiente guía de estudio tiene como objetivo practicar y ejercitar los contenidos que hemos visto en clase.

¡Vamos con todo!



Tabla de contenidos

Actividad guiada: Sitio Venta de Tickets para Eventos	2
Estructura del Proyecto	4
Creación de Proyecto	5
Creación Vistas JSP y Clases Java	7
Framework Bootstrap	10
Código Fuente Java	11
Includes JSP	17
Vistas JSP	20
Ejecución del Proyecto	29
Actividad guiada - Agrega Detalle de Evento	33
Actividad guiada - Agrega Mapa de Evento	40
Preguntas de proceso	46
Preguntas de cierre	46
Referencias bibliográficas	46



¡Comencemos!



Actividad guiada: Sitio Venta de Tickets para Eventos

Estimados alumnos, sean muy bienvenidos!, en esta actividad vamos a poner en práctica los aprendizajes previos relacionados con el paso de información vía formulario en proyectos Web Dinámicos.

Para lograrlo, necesitarás aplicar todo lo aprendido en los bloques 8 y 9 de esta unidad, (te aconsejamos mantener a mano dicho material en caso de cualquier duda).

En este ejercicio vamos a desarrollar un nuevo proyecto Java Web Dinámico, en el cual tendremos un formulario web desde el cual podremos reservar tickets para eventos de diverso tipo.

Para esto nos apoyaremos de las capacidades que nos entrega la programación orientada a objetos (POO), creando objetos de tipo “Evento” y manejo de los mismos por listas <List>. Todo esto en conjunto con clases Java especializadas y los respectivos JSP para el traspaso de información.

Aplicando los conceptos y herramientas aprendidas hasta ahora, generamos lo siguiente:

- Un proyecto Java Web Dinámico llamado “**Actividad11**”.
- Un JSP llamado “**index.jsp**”, encargado de la presentación para captura de valores.
- Vinculación POST con JSP “**procesa.jsp**”.
- Creación de paquetes y clases Java necesarias.
- Generación de respuesta con valores de salida de acuerdo a la selección del evento.

Adicionalmente, tendremos una visión de estructura HTML con marco de trabajo bootstrap y separación de estructura con includes.

¡Suenan genial!

Venta de Tickets para Eventos

Tickets para Eventos

Eventos Disponibles (4)

 Metallica Descripción Uno CL\$ 10000 ☐ Ver Detalle Evento	 Fiestas Patrias Descripción Dos CL\$ 20000 ☐ Ver Detalle Evento	 Iron Maiden Descripción Tres CL\$ 30000 ☐ Ver Detalle Evento	 Delfin Hasta el Fin! Descripción Cuatro CL\$ 40000 ☒ Ver Detalle Evento
---	---	--	--

Información de Venta

Nombre	Apellido	
Marcelo	Salas Melinao	
Dirección	Número	Cantidad de Tickets
Av. 15 Norte	487	1 Ticket(s)
Enviar Solicitud de Compra		

Imagen 1. Vista "index.jsp"
Fuente: Desafío Latam

Venta de Tickets para Eventos

Tickets para Eventos

Detalle Solicitud Venta Ticket: **Metallica**

#	Item	Valor
1	Nombre	Marcelo
2	Apellido	Salas Melinao
3	Dirección	Av. 15 Norte
4	Numero	487
5	Cantidad Tickets	1
6	Valor a Pagar	CL\$ 10000

Desafío Latam - 2022

Imagen 2. Vista “procesa.jsp”

Fuente: Desafío Latam

Estructura del Proyecto

Para comenzar vamos a definir una estructura de proyecto separada para los elementos propios de página y los que son generales del sistema.

En este sentido podemos ver y encontrar elementos generales como header y footer que son parte de la construcción de cada JSP, por tanto, los podemos agregar a modo de “includes”.

Todos estos elementos anexos forman parte de la estructura de /assets del sitio, tal y como lo son los .JS, Imágenes y cualquier otro elemento anexo.

La estructura en consecuencia queda del siguiente modo:

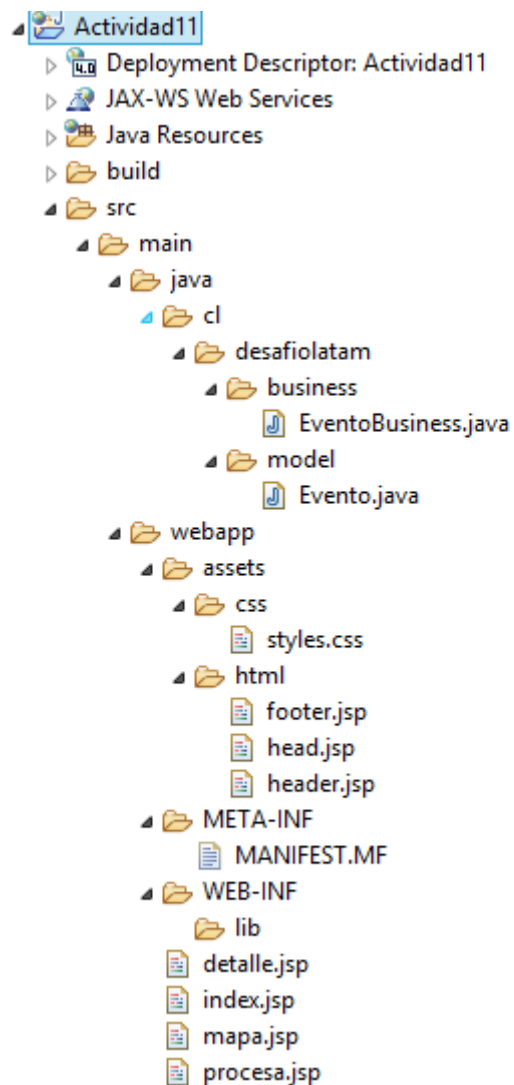


Imagen 3. Estructura de carpetas del proyecto
Fuente: Desafío Latam

Creación de Proyecto

Para crear el proyecto desde Eclipse debemos ir al menú **"File"**, luego seleccionar **"New"** y finalmente elegir desde el listado la opción **"Dynamic Web Project"**

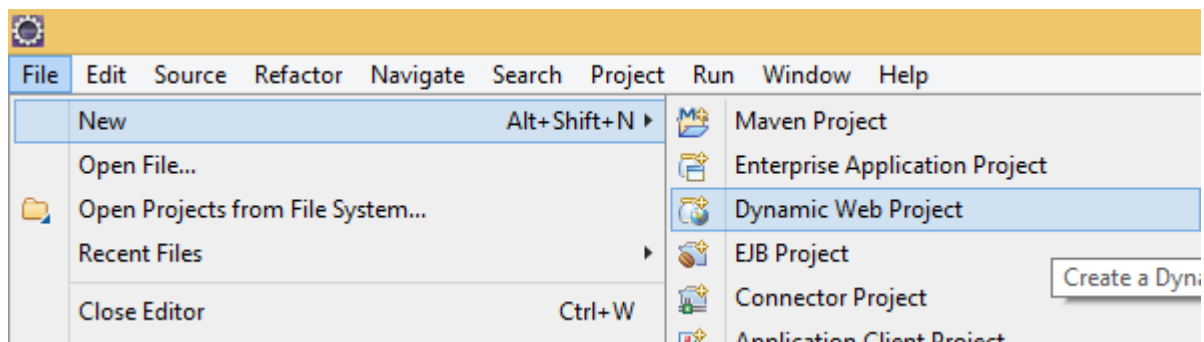


Imagen 4. Creación de un nuevo proyecto

Fuente: Desafío Latam

Una vez seleccionado, eclipse desplegará el cuadro de creación de un nuevo proyecto. Desde acá debemos indicar el nombre que deseamos usar para el mismo. En nuestro caso le daremos por nombre “**Actividad11**”, tal como se muestra en la siguiente imagen:

Project name:

Project location

☒ Use default location

Location:

Target runtime

Dynamic web module version

Configuration

A good starting point for working with Apache Tomcat v9.0 runtime. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

EAR project name:

Working sets

☐ Add project to working sets

Working sets:

Imagen 5. Creación de un nuevo proyecto
Fuente: Desafío Latam

Creación Vistas JSP y Clases Java

El siguiente paso es crear cada uno de los elementos que utilizaremos en nuestro proyecto, para esto debemos crear las vistas JSP, las clases java especializadas, sus métodos e incluir los elementos anexos del sitio, esto sería:

Creación de JSP

- /index.jsp
- /procesa.jsp
- /assets/html/footer.jsp
- /assets/html/head.jsp
- /assets/html/header.jsp
- /assets/css/styles.css

Java

- /cl/desafiolatam/model/Evento.java
- /cl/desafiolatam/business/EventoBusiness.java

Para esto, como siempre, el proceso es desde Eclipse, "File", "New", "JSP File"

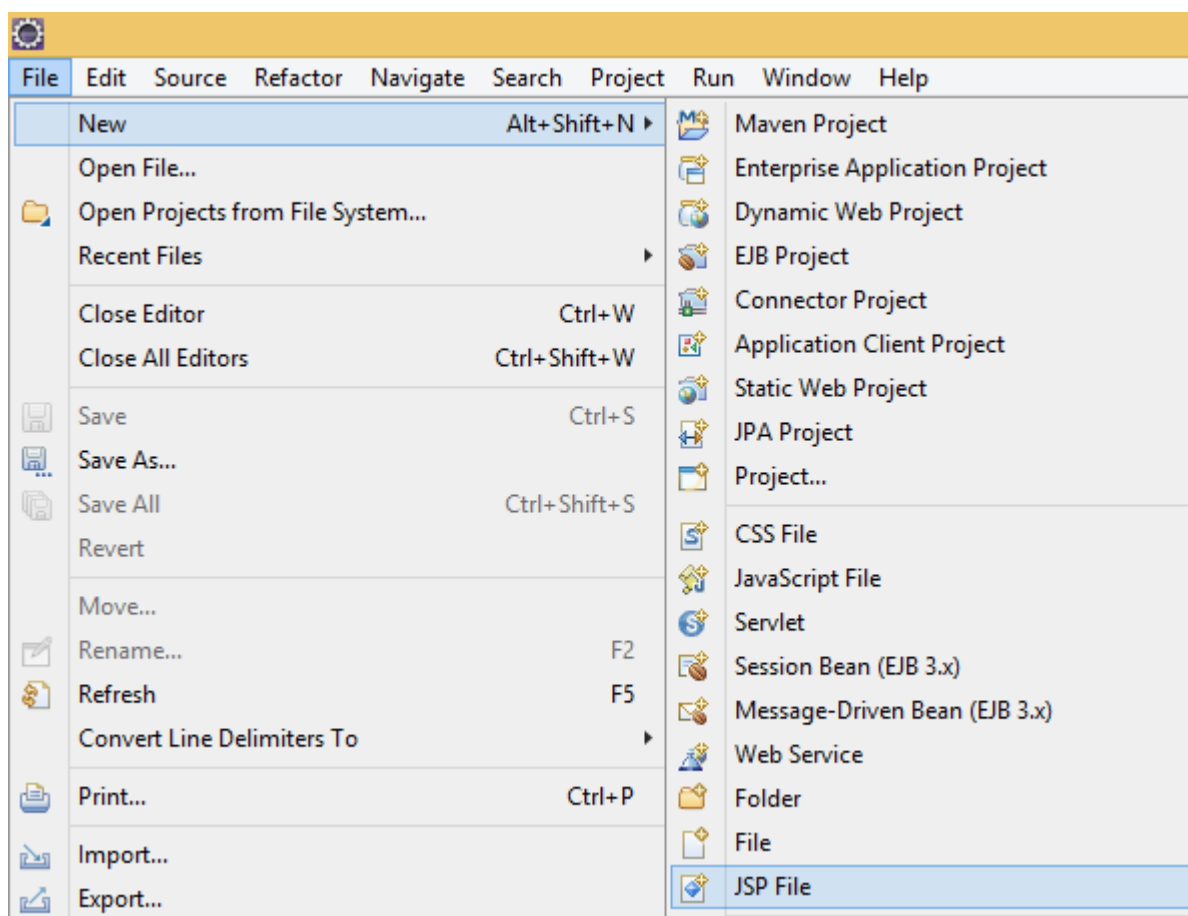


Imagen 6. Creación de vista JSP

Fuente: Desafío Latam

y para las clases Java **clic derecho sobre el proyecto, "New", "Class"**.

Importante:



Recordar que vamos a usar 2 paquetes para las clases java **“cl.desafiolatam.model”** y **“cl.desafiolatam.business”**, los cuales se deben también crear desde clic derecho sobre el proyecto , **“New”**, **“Package”**.

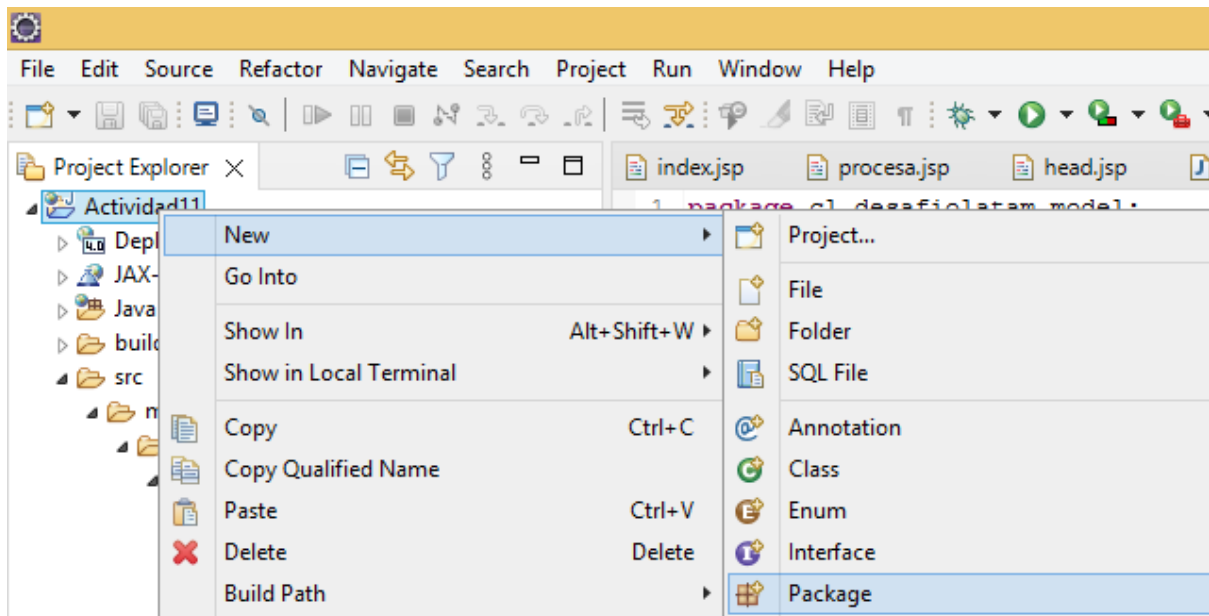


Imagen 7. Creación de paquetes

Fuente: Desafío Latam

Luego se presentará el cuadro de creación de paquetes, donde ingresamos los nombres respectivos **“cl.desafiolatam.model”**, y luego **“cl.desafiolatam.business”**.

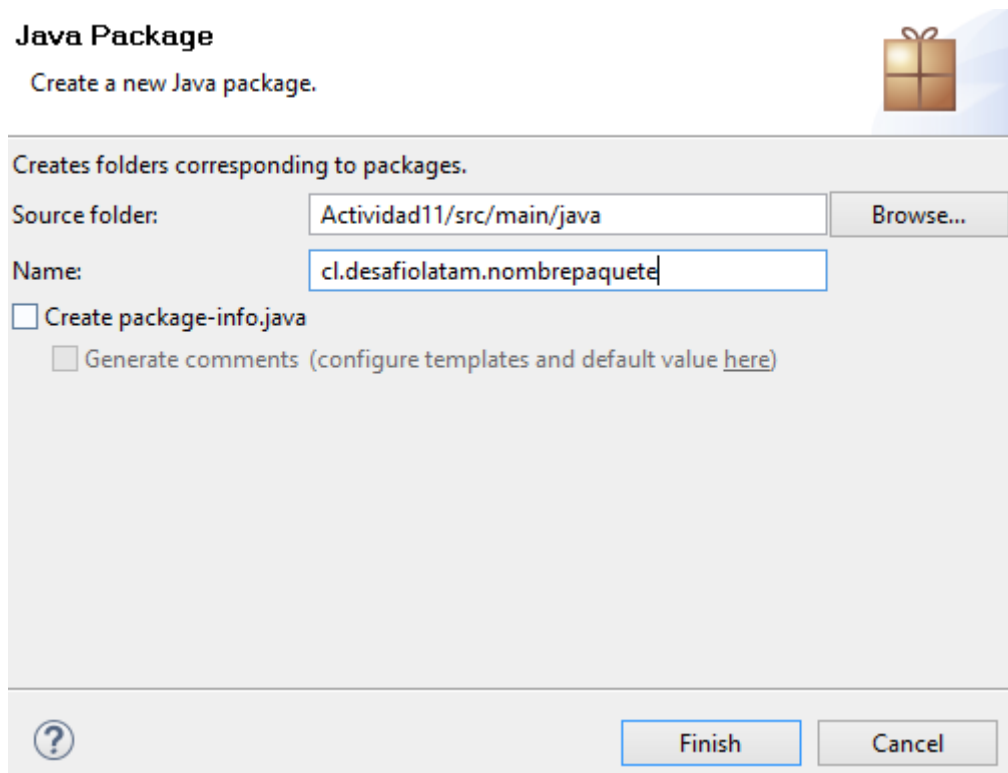


Imagen 8. Creación de paquetes
Fuente: Desafío Latam

¡Con esto ya tenemos nuestra estructura lista, excelente!

Framework Bootstrap

Como indicamos anteriormente, aprovecharemos las bondades que nos entregan los marcos de trabajo actuales para todo lo que refiere a la visualización de sistemas (capa front), y en este sentido anexamos Bootstrap al proyecto para hacerlo más atractivo y también así adaptativo a diferentes tipos de dispositivos (PC, Tablets y Celulares).

Para esto, por medio del sitio de **Bootstrap** usaremos parte de los templates existentes y lo adaptamos a la salida deseada.

Esto desde el sitio <https://startbootstrap.com/template/shop-homepage>, botón “**Free Download**” (de igual modo en la carpeta de fuentes encontrarán el proyecto 100% funcional).

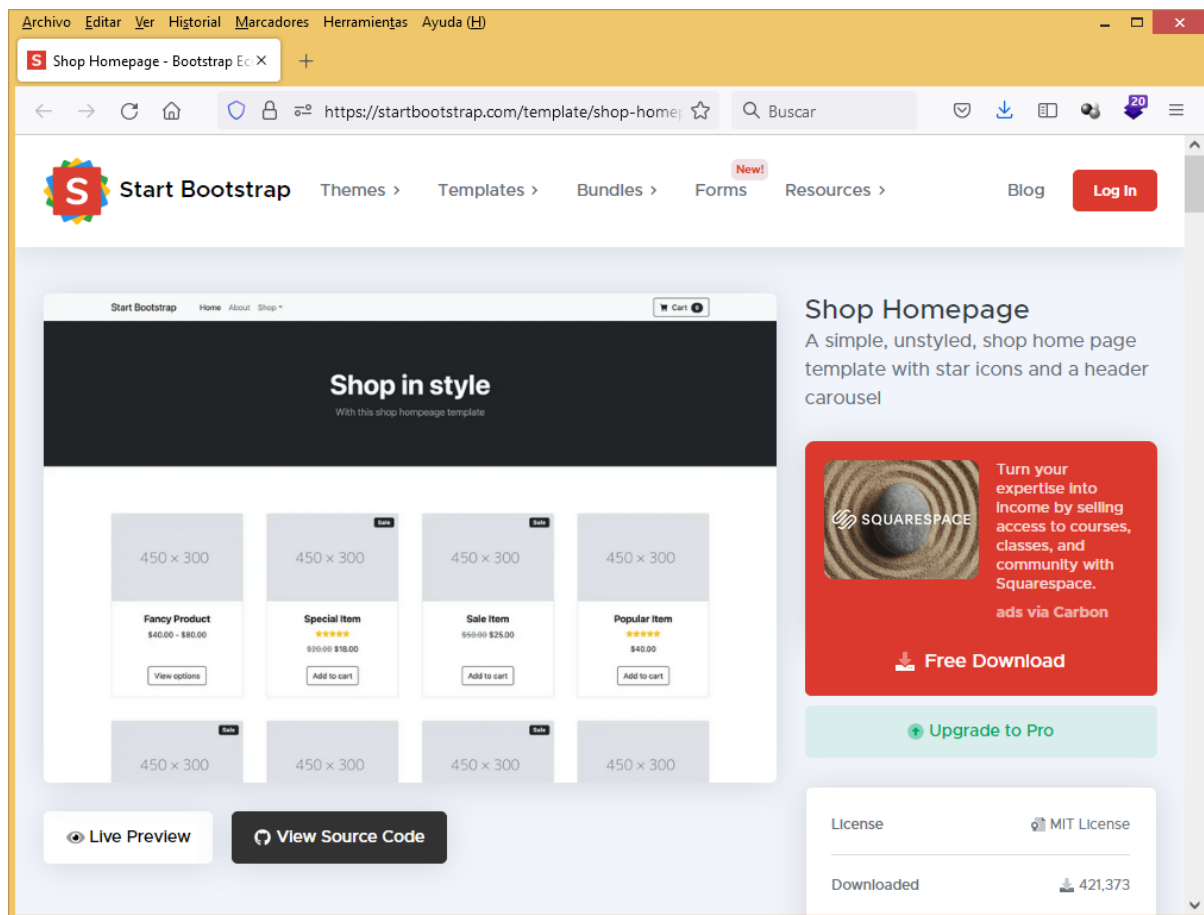


Imagen 9. Framework Bootstrap
Fuente: Desafío Latam

Código Fuente Java

Ahora ya tenemos nuestra estructura y también el marco de trabajo, por tanto, ¡manos al código!

Desarrollaremos una estructura Java basada en objetos de tipo "Evento", el cual nos permitirá almacenar toda la información necesaria para poder desplegar y consultar sus datos. Los atributos del objeto serán los siguientes:

Atributos de clase

- private int id;
- private String nombre;
- private String descripcion;
- private String imagen;
- private int precio;

- private String lugar;
- private String mapa;

En este objeto guardaremos el identificador de evento que luego será la clave para hacer viajar la info y seleccionar todos los datos del mismo (entre ellos el nombre, su descripción, imagen y otros).

Por tanto, debemos crear las siguientes clases en los paquetes definidos, del siguiente modo:

Clase Evento / (Paquete: cl.desafiolatam.model)

```
package cl.desafiolatam.model;

/**
 * -----
 * -----
 * Clase Evento
 *
 * @author: Autor <autor@desafiolatam.com>
 * @version: 23/08/2022
 */
public class Evento {

    // Atributos de clase
    private int id;
    private String nombre;
    private String descripcion;
    private String imagen;
    private int precio;
    private String lugar;
    private String mapa;

    // Getters y Setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }
}
```

```
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getDescripcion() {
    return descripcion;
}

public void setDescripcion(String descripcion) {
    this.descripcion = descripcion;
}

public String getImagen() {
    return imagen;
}

public void setImagen(String imagen) {
    this.imagen = imagen;
}

public int getPrecio() {
    return precio;
}

public void setPrecio(int precio) {
    this.precio = precio;
}

public String getLugar() {
    return lugar;
}

public void setLugar(String lugar) {
    this.lugar = lugar;
}

public String getMapa() {
    return mapa;
}

public void setMapa(String mapa) {
    this.mapa = mapa;
}
```

```
}  
  
}
```

Clase EventoBusiness / (Paquete: cl.desafiolatam.business)

La clase **EventoBusiness**, tendrá toda la lógica de negocio (vale decir los procesos de creación, llenado, búsqueda, selección cálculos y otros hacia el objeto). Luego en la medida que avancemos en el curso veremos que toda esta data la podremos rescatar también así de fuentes de datos como base de datos o similares, donde esta estructura es la base.

```
package cl.desafiolatam.business;  
  
import java.util.ArrayList;  
import java.util.List;  
  
import cl.desafiolatam.model.*;  
  
public class EventoBusiness {  
  
    /**  
     * -----  
     * -----  
     * Método setea y obtiene los eventos existentes en una lista de  
objetos de tipo  
     * evento  
     *  
     * @param -  
     * @return List<Evento>  
     * @author: Autor <autor@desafiolatam.com>  
     * @version: 23/08/2022  
     */  
    public List<Evento> getEventos() {  
  
        // Inicializacion de lista de eventos  
        ArrayList<Evento> eventos = new ArrayList<Evento>();  
  
        // Info Evento 1  
        Evento evento1 = new Evento();  
        evento1.setId(1);  
        evento1.setNombre("Metallica");  
        evento1.setDescripcion("Descripción Uno");  
    }  
}
```

```
evento1.setImagen(  
    "https://media.gettyimages.com/photos/kirk-hammett-  
lars-ulrich-james-hetfield-and-robert-trujillo-from-at-picture-  
id130699422?s=2048x2048");  
evento1.setPrecio(10000);  
evento1.setLugar("Estadio Nacional");  
evento1.setMapa(  
  
"https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3328.411793967857  
!2d-70.61286488437952!3d-  
33.46462808076988!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x96  
62cfed639ab813%3A0xb92488060556a1df!2sEstadio%20Nacional%20Julio%20Mart%  
C3%ADnez%20Pr%C3%A1danos!5e0!3m2!1ses!2scl!4v1661265818941!5m2!1ses!2scl  
");  
  
// Info Evento 2  
Evento evento2 = new Evento();  
evento2.setId(2);  
evento2.setNombre("Fiestas Patrias");  
evento2.setDescripcion("Descripción Dos");  
evento2.setImagen("https://img.freepik.com/vector-gratis/fondo-  
plano-fiestas-patrias-chile_23-2149539845.jpg");  
evento2.setPrecio(20000);  
evento2.setLugar("parque O'higgins");  
evento2.setMapa(  
  
"https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3328.411793967857  
!2d-70.61286488437952!3d-  
33.46462808076988!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x96  
62cfed639ab813%3A0xb92488060556a1df!2sEstadio%20Nacional%20Julio%20Mart%  
C3%ADnez%20Pr%C3%A1danos!5e0!3m2!1ses!2scl!4v1661265818941!5m2!1ses!2scl  
");  
  
// Info Evento 3  
Evento evento3 = new Evento();  
evento3.setId(3);  
evento3.setNombre("Iron Maiden");  
evento3.setDescripcion("Descripción Tres");  
evento3.setImagen(  
    "url_imagen");  
evento3.setPrecio(30000);  
evento3.setLugar("Estadio Monumental");  
evento3.setMapa(  

```

```
"https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3328.411793967857!2d-70.61286488437952!3d-33.46462808076988!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x9662cfed639ab813%3A0xb92488060556a1df!2sEstadio%20Nacional%20Julio%20Mart%C3%ADnez%20Pr%C3%A1danos!5e0!3m2!1ses!2scl!4v1661265818941!5m2!1ses!2scl");
```

```
// Info Evento 4
Evento evento4 = new Evento();
evento4.setId(4);
evento4.setNombre("Delfín Hasta el Fin!");
evento4.setDescripcion("Descripción Cuatro");
evento4.setImagen(
    "https://www.prensalibre.com/wp-content/uploads/2021/11/Delfin-1.jpg?quality=52&w=760&h=430&crop=1");
evento4.setPrecio(40000);
evento4.setLugar("Centro de Eventos Amanda");
evento4.setMapa(
```

```
"https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3328.411793967857!2d-70.61286488437952!3d-33.46462808076988!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x9662cfed639ab813%3A0xb92488060556a1df!2sEstadio%20Nacional%20Julio%20Mart%C3%ADnez%20Pr%C3%A1danos!5e0!3m2!1ses!2scl!4v1661265818941!5m2!1ses!2scl");
```

```
// Agrega Eventos a lista
eventos.add(evento1);
eventos.add(evento2);
eventos.add(evento3);
eventos.add(evento4);

// Retorno de lista de eventos
return eventos;
}
```

```
/**
```

```
* -----
```

```
* Método que entrega el objeto evento segun el id de busqueda
```

```
*
```

```
* @param Evento evento
```

```
* @return Evento evento
```

```
* @author: Autor <autor@desafiolatam.com>
```

```
* @version: 23/08/2022
*/
public Evento getEventobyID(Evento evento) {

    List<Evento> listaEventos = getEventos();

    for (int x = 0; x < listaEventos.size(); x++) {

        if (listaEventos.get(x).getId() == evento.getId()) {
            evento = listaEventos.get(x);
        }
    }

    // Retorno de Evento
    return evento;
}

/**
 * -----
 * Método retorna el valor a pagar de un evento segun el valor y
cantidad de
 * tickets seleccionados
 *
 * @param int valorEvento, int cantidadTickets
 * @return int valor
 * @author: Autor <autor@desafiolatam.com>
 * @version: 23/08/2022
 */
public int calculaValorCompra(int valorEvento, int cantidadTickets)
{
    int valor = valorEvento * cantidadTickets;
    return valor;
}
}
```

Includes JSP

Tal como señalamos al comienzo, y como parte de cualquier desarrollo de un sistema real, separaremos de las vistas propias los elementos generales del sitio (entre ellos el footer, header y otros elementos compartidos).

Esto tiene una lógica y propósito claro, es cual es desagrupar para una mejor mantención de código a futuro, por esto cualquier cambio de header o footer se realizará en un solo lugar y se replica a todo el sistema.

Para esto vamos a incluir el código de cada jsp de “assets” del siguiente modo:

1. **Header:** Almacenará toda el HTML relacionado al encabezado (logo, barra de navegación, etc.).
2. **Footer:** Almacenará los elementos de pie de sitio.
3. **Head:** Almacenará el contenido de <head>, entre ellas los includes hacia el marco de trabajo Bootstrap por medio JS y CSS.

Finalmente, estos JSP de assets se incorporarán en nuestros JSP principales (index.jsp, procesa.jsp), por medio de los includes respectivos.

```
<%@include file="assets/html/head.jsp"%>
<%@include file="assets/html/header.jsp"%>
<%@include file="assets/html/footer.jsp"%>
```

Head - /assets/html/head.jsp

```
<head>
<meta charset="utf-8" />
<meta name="viewport"
  content="width=device-width, initial-scale=1, shrink-to-fit=no" />
<meta name="description" content="" />
<meta name="author" content="" />
<title>Desafio Latam - Eventos</title>
<!-- Favicon-->
<link rel="icon" type="image/x-icon" href="assets/favicon.ico" />
<!-- Bootstrap icons-->
<link
  href="https://cdn.jsdelivr.net/npm/bootstrap-
  icons@1.5.0/font/bootstrap-icons.css"
  rel="stylesheet" />
<!-- Core theme CSS (includes Bootstrap)-->
<link href="assets/css/styles.css" rel="stylesheet" />
</head>
```

Header - /assets/html/header.jsp

```
<!-- Navigation-->
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container px-4 px-lg-5">
    <a class="navbar-brand" href="index.jsp"></
a>
    <button class="navbar-toggler" type="button" data-bs-
toggle="collapse"
      data-bs-target="#navbarSupportedContent"
      aria-controls="navbarSupportedContent" aria-expanded="false"
      aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarSupportedContent">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0 ms-lg-4">
        <li class="nav-item"><a class="nav-link active"
          href="index.jsp">Inicio</a></li>
        <li class="nav-item"><a class="nav-link"
          href="https://desafiolatam.com/politica-de-
calidad/">Política
          de Calidad</a></li>
        <li class="nav-item dropdown"><a
          class="nav-link dropdown-toggle"
          id="navbarDropdown" href="#"
          role="button" data-bs-toggle="dropdown" aria-
expanded="false">Desafío
          Latam</a>
          <ul class="dropdown-menu" aria-
labelledby="navbarDropdown">
            <li><a class="dropdown-item"
              href="https://desafiolatam.com/">Desafio
                Latam</a></li>
            <li><hr class="dropdown-divider" /></li>
            <li><a class="dropdown-item"
              href="https://desafiolatam.com/comunidad/">Comunidad Desafío
                Latam</a></li>
          </ul></li>
      </ul>
    </div>
  </div>
```

```
</nav>
<!-- Header-->
<header class="bg-dark py-5">
  <div class="container px-4 px-lg-5 my-5">
    <div class="text-center text-white">
      <h1 class="display-4 fw-bolder">Venta de Tickets para
Eventos</h1>
      <p class="lead fw-normal text-white-50 mb-0">Tickets para
Eventos</p>
    </div>
  </div>
</header>
```

Footer- /assets/html/footer.jsp

```
<!-- Footer-->
<footer class="py-5 bg-dark">
  <div class="container">
    <p class="m-0 text-center text-white">Desafío Latam - 2022</p>
  </div>
</footer>
<!-- Bootstrap core JS-->
<script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"></script>
<!-- Core theme JS-->
<script src="js/scripts.js"></script>
```

Vistas JSP

¡Excelente!, hasta acá ya tenemos toda nuestra lógica de negocio realizada. Tenemos nuestras clases Java (Modelo y Business), los assets del sitio (bootstrap con separación de header, footer) y la lógica de estructura.

Por tanto, ya solo nos queda crear los JSP de vista con llamados a los procesos de visualización de formulario y finalmente la página JSP de recepción de datos del mismo.

En este sentido tendremos lo siguiente:

1. **index.jsp**: Página que contiene el inicio del sistema, incluye el formulario de envío de datos con la selección del evento, que a su vez trae los datos desde la lista de objetos "Evento" seteada en la clase "EventoBusiness".
2. **procesa.jsp**: Página que contiene las acciones de despliegue y salida de datos entre ellas recepción de request y cálculo de valor.

Importante:



Recordar que como se señalaba en presentaciones anteriores tenemos múltiples formas de poder desarrollar una tarea.

En este caso podremos usar arreglos, listas, clases y cualquier otro tipo de elemento del paradigma de Programación Orientada a Objetos (POO) y del lenguaje para nuestros elementos.

De igual forma podríamos usar alguna librería como JSTL o similar, en este caso de ejemplo lo hemos realizado de la manera tradicional y más simple, para así tener un control directo de los elementos y sea más entendible.

Index.jsp

Como podemos ver, index.jsp contiene todo el proceso de visualización del formulario. Incluye las etiquetas HTML del mismo <form> y cada elemento que lo compone.

Podemos ver la estructura de lógica de negocio aplicada en los llamados a las clases java, desde donde extraemos la información de Eventos por medio de una lista, la cual es recorrida por un ciclo iterativo:

```
// Librerías Requeridas
EventoBusiness eventob = new EventoBusiness();

// listado de Evento
List<Evento> listaEventos = eventob.getEventos();
```

De esta manera podemos agregar a futuro cualquier evento sin tocar la capa de visualización, acá tenemos aplicado el concepto de front y backend de un sistema.

Luego la información se envía vía POST a destino, todo seteado en el formulario por medio de:

```
<form method="post" action="procesa.jsp">
```

Y realizado desde el botón submit respectivo:

```
<button type="submit" class="btn btn-primary">Enviar Solicitud de Compra</button>
```

```
<%@page import="java.util.List"%>
<%@page import="cl.desafiolatam.model.Evento"%>
<%@page import="java.util.ArrayList"%>
<%@page import="cl.desafiolatam.business.EventoBusiness"%>
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%
// Librerias Requeridas
EventoBusiness eventob = new EventoBusiness();

// listado de Evento
List<Evento> listaEventos = eventob.getEventos();
%>

<!DOCTYPE html>
<html lang="es">
<%@include file="assets/html/head.jsp"%>
<body>

    <%@include file="assets/html/header.jsp"%>

    <form method="post" action="procesa.jsp">

        <!-- Section-->
        <section class="py-5">
            <div class="container px-4 px-lg-5 mt-5">

                <div class="row justify-content-center">
                    <h1>
                        Eventos Disponibles
                    (<%=listaEventos.size()%>)
                    </h1>
                    <hr/ >
                </div>

                <div
                    class="row gx-4 gx-lg-5 row-cols-2 row-cols-md-3
row-cols-xl-4 justify-content-center">

                    <%
                    for (int x = 0; x < listaEventos.size(); x++) {
                    %>

                    <div class="col mb-5">
```

```

        <div class="card h-100">
            " />
            <div class="card-body p-4">
                <div class="text-center">
                    <h5 class="fw-
bolder"><%=listaEventos.get(x).getNombre()%></h5>

<p><%=listaEventos.get(x).getDescripcion()%></p>
                    CL$

<%=listaEventos.get(x).getPrecio()%>
                    </div>
                </div>
                <input type="radio" id="html"
name="idevento"

value="<%=listaEventos.get(x).getId()%>" checked>
                <div class="card-footer p-4 pt-0
border-top-0 bg-transparent">
                    <div class="text-center">
                        <a class="btn btn-
outline-dark mt-auto"

href="detalle.jsp?idevento=<%=listaEventos.get(x).getId()%>">Ver
                        Detalle Evento</a>
                    </div>
                </div>
            </div>
        </div>
    </div>
    <%
    }
    %>

</div>
</div>
</section>

<!-- Formulario -->
<section>

```

```
<div class="container px-4 px-lg-5 mt-5">

    <h1>Información de Venta</h1>
    <hr />

    <div class="row g-3">
        <div class="col-md-6">
            <label for="inputEmail4" class="form-
label">Nombre</label> <input
                                type="text" class="form-control"
id="nombre" name="nombre"
                                value="Marcelo">
        </div>
        <div class="col-md-6">
            <label for="inputPassword4" class="form-
label">Apellido</label> <input
                                type="text" class="form-control"
id="apellido" name="apellido"
                                value="Salas Melinao">
        </div>

        <div class="col-md-6">
            <label for="inputCity" class="form-
label">Dirección</label> <input
                                type="text" class="form-control"
id="direccion" name="direccion"
                                value="Av. 15 Norte">
        </div>

        <div class="col-md-2">
            <label for="inputZip" class="form-
label">Número</label> <input
                                type="text" class="form-control"
id="numero" name="numero"
                                value="487">
        </div>

        <div class="col-md-4">
            <label for="inputState" class="form-
label">Cantidad de
                                Tickets</label> <select
id="cantidad" name="cantidad" class="form-select">
                <%
                for (int x = 0; x < 50; x++) {
```

```
1%> Ticket(s)
                                %>
                                <option value="<%=x + 1%>"><%=x +
                                </option>
                                <%
                                }
                                %>

                                </select>
                                </div>

                                <div class="col-12">
                                <hr />
                                <button type="submit" class="btn btn-
primary">Enviar
                                Solicitud de Compra</button>
                                </div>
                                </div>

                                </div>

                                </section>

                                <br /> <br />

                                </form>

                                <%@include file="assets/html/footer.jsp"%>

                                </body>
                                </html>
```

procesa.jsp

Finalmente, y ya con nuestra vista realizada, solo nos queda crear nuestra página JSP con el proceso de recepción de información.

Este proceso tomará los datos y evaluará los mismos para generar la salida final por pantalla al usuario, presentando una tabla de datos con el resumen de ticket y el cálculo de valor final a pagar.

Importante:

Recordar que esto último, forma parte de los llamados a las funciones Java respectivas de **“EventoBusiness”**.

Con esto queda del siguiente modo la lógica aplicada:

Se realiza el proceso de envío de datos desde la página “index.jsp”, luego en la página “procesa.jsp”, se captura la información de cada uno de los inputs del formulario (como el nombre, apellido, dirección y otros).

```
String nombreComprador = request.getParameter("nombre");  
String apellidoComprador = request.getParameter("apellido");  
String direccionComprador = request.getParameter("direccion");  
int numeroComprador = Integer.parseInt(request.getParameter("numero"));  
int cantidadTickets = Integer.parseInt(request.getParameter("cantidad"));  
int idEvento = Integer.parseInt(request.getParameter("idevento"));
```

Se crea y define el evento con su ID respectivo, para luego obtener sus datos particulares desde la función **“getEventobyID”** que se encuentra en la clase **“EventoBusiness”** instanciado en el objeto **“eventob”**.

```
Evento evento = new Evento();  
evento.setId(idEvento);  
evento = eventob.getEventobyID(evento);
```

Finalmente, se realiza el llamado al método para cálculo de valor final a pagar, donde se entrega el precio del evento * la cantidad de tickets a comprar desde **“calculaValorCompra”**

```
int valorPagar = eventob.calculaValorCompra(evento.getPrecio(), cantidadTickets);
```

Con esto se despliega información por medio de las etiquetas **<%= %>** respectivas.

```
<%@page import="cl.desafiolatam.model.Evento"%>  
<%@page import="cl.desafiolatam.business.EventoBusiness"%>  
<%@ page language="java" contentType="text/html; charset=UTF-8"  
    pageEncoding="UTF-8"%>  
  
<%  
// Recepcion valores dede request  
String nombreComprador = request.getParameter("nombre");  
String apellidoComprador = request.getParameter("apellido");  
String direccionComprador = request.getParameter("direccion");  
int numeroComprador = Integer.parseInt(request.getParameter("numero"));  
int cantidadTickets =
```

```
Integer.parseInt(request.getParameter("cantidad"));
int idEvento = Integer.parseInt(request.getParameter("idevento"));

// Librerias requeridas
EventoBusiness eventob = new EventoBusiness();

// inicializa Objeto Evento
Evento evento = new Evento();
evento.setId(idEvento);

// Busca y recoge informacion de evento seleccionado
evento = eventob.getEventobyID(evento);

// Obtiene el valor a pagar por la cantidad de tickets seleccionados
int valorPagar = eventob.calculaValorCompra(evento.getPrecio(),
cantidadTickets);
%>
<!DOCTYPE html>
<html lang="es">
<%@include file="assets/html/head.jsp"%>
<body>

    <%@include file="assets/html/header.jsp"%>

    <form method="post" action="procesa.jsp">

        <!-- Section-->
        <section class="py-5">
            <div class="container px-4 px-lg-5 mt-5">

                <div class="row justify-content-center">
                    <h1>
                        Detalle Solictud Venta Ticket:
                    </h1>
                    <hr/ >
                </div>
            </div>

            <div class="container px-4 px-lg-5 mt-5">

                <table class="table">
                    <thead>
                        <tr>
```

```
        <th scope="col">#</th>
        <th scope="col">Item</th>
        <th scope="col">Valor</th>
    </tr>
</thead>
<tbody>
    <tr>
        <th scope="row">1</th>
        <td>Nombre</td>
        <td><%=nombreComprador%></td>
    </tr>
    <tr>
        <th scope="row">2</th>
        <td>Apellido</td>
        <td><%=apellidoComprador%></td>
    </tr>
    <tr>
        <th scope="row">3</th>
        <td>Dirección</td>
        <td><%=direccionComprador%></td>
    </tr>
    <tr>
        <th scope="row">4</th>
        <td>Numero</td>
        <td><%=numeroComprador%></td>
    </tr>
    <tr>
        <th scope="row">5</th>
        <td>Cantidad Tickets</td>
        <td><%=cantidadTickets%></td>
    </tr>
    <tr>
        <th scope="row">6</th>
        <td>Valor a Pagar</td>
        <td><strong>CL$
<%=valorPagar%></strong></td>
    </tr>
</tbody>
</table>
```

```
        </div>

    </section>

    <br /> <br />

</form>

<%@include file="assets/html/footer.jsp"%>

</body>
</html>
```

Ejecución del Proyecto

Con todo lo anterior, ya podemos correr nuestro proyecto, para esto, como siempre desde el nombre del mismo, damos con el botón derecho y seleccionamos **"Run As"**, **"Run on Server"**.

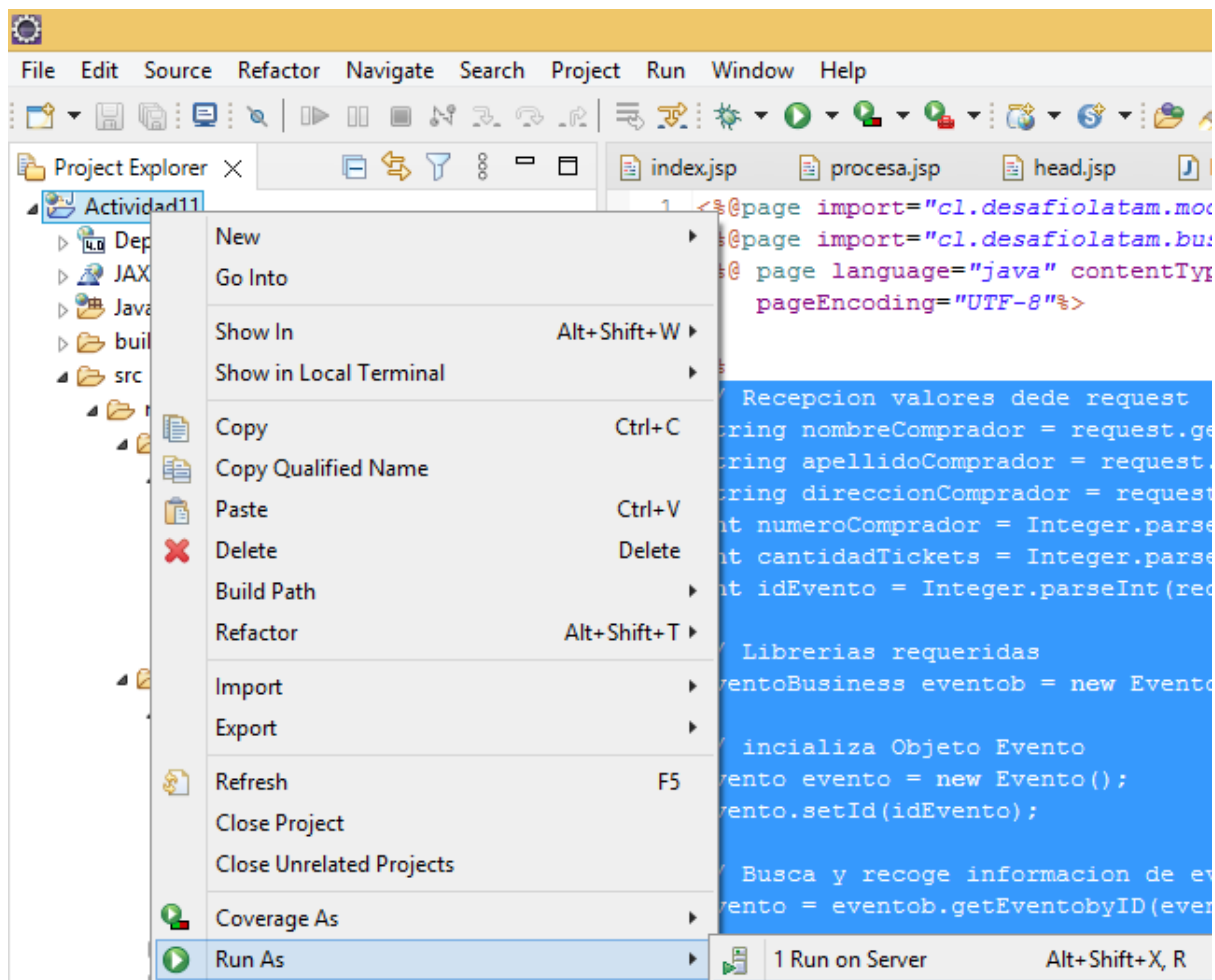


Imagen 10. Ejecución del Proyecto

Fuente: Desafío Latam

Acá se desplegará el index.jsp con la información de los eventos y el formulario creado para la captura de información.

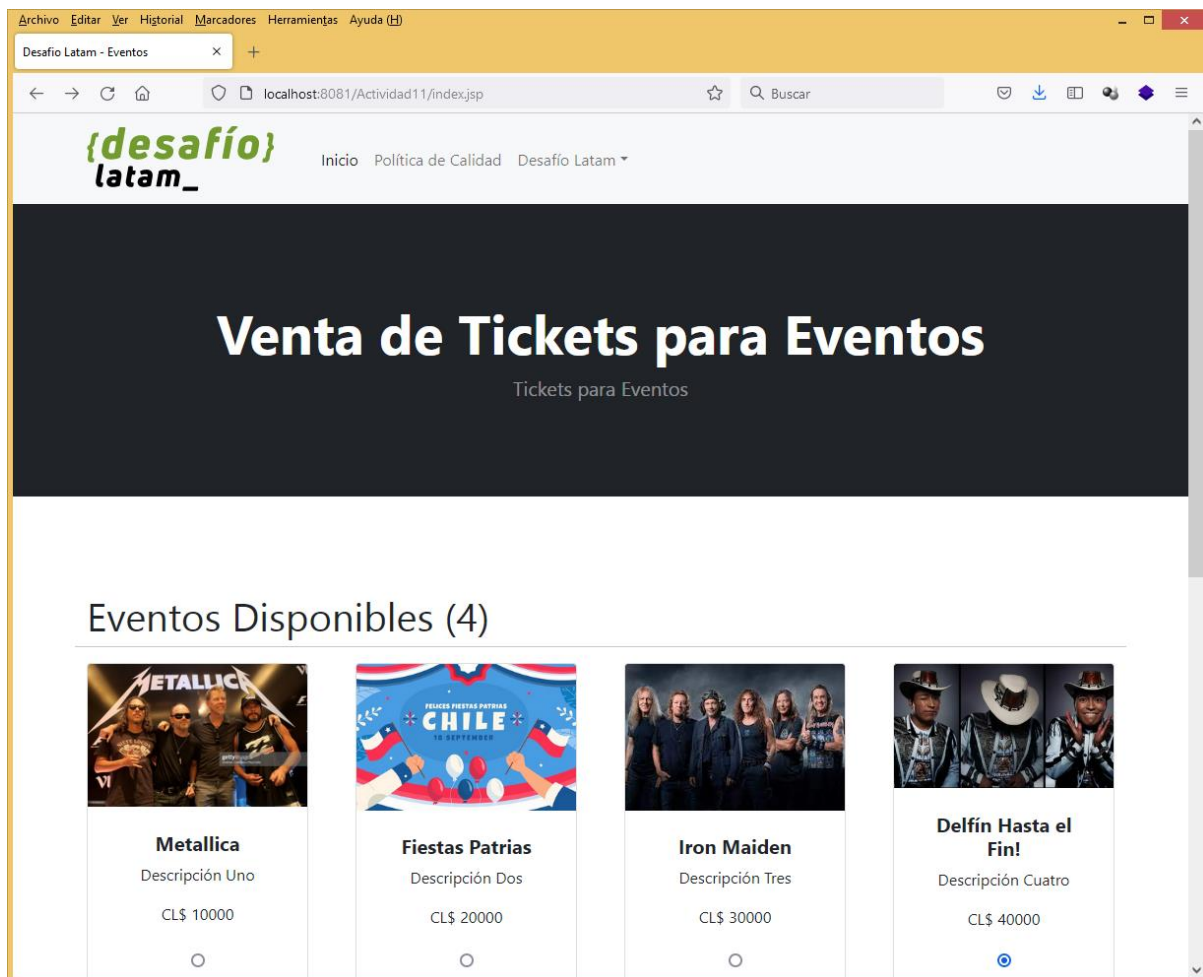


Imagen 11. Vista index.jsp desde el navegador
Fuente: Desafío Latam

También podemos ver el JSP con la salida del proceso de captura de datos, donde se presenta la siguiente tabla.

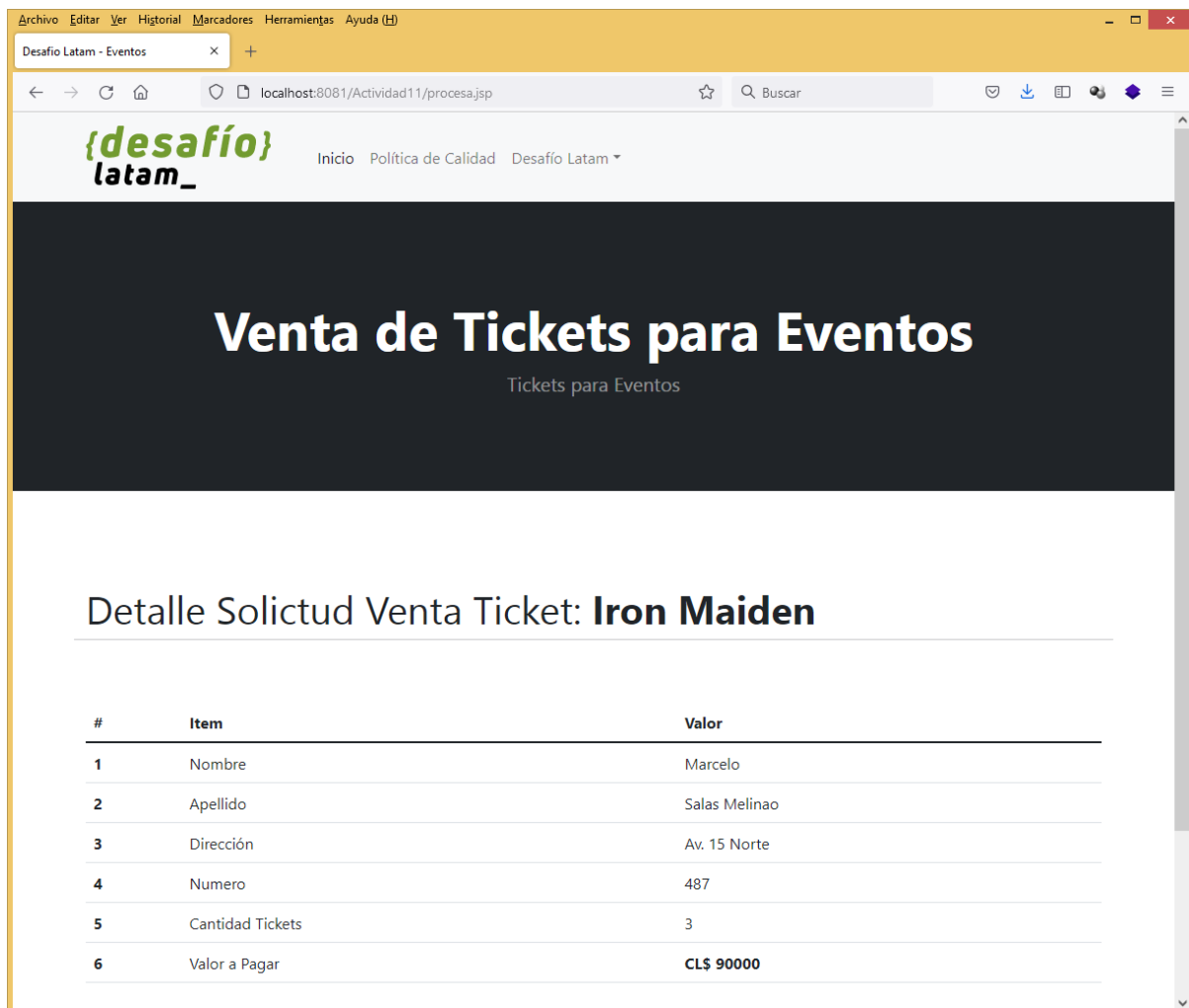


Imagen 12. Vista procesa.jsp desde el navegador
Fuente: Desafío Latam

¡Genial!, ya tenemos nuestro sistema de tickets funcionando y corriendo,
¡felicitaciones!



Actividad guiada - Agrega Detalle de Evento

Estimado alumno que tal!, hasta acá ya tenemos nuestro sistema con presentación de eventos y selección de tickets perfectamente funcionando.

Para esta segunda actividad vamos a agregar una nueva característica, la cual es crear una página JSP de detalle de evento, con esto quien visite el sitio también podrá ver la ficha del mismo.

Para esto vamos a crear en la estructura el siguiente archivo

/detalle.jsp

Tal y como hemos realizado las visitas anteriores (index.jsp y procesa.jsp), usamos el mismo diseño usado con bootstrap y assets ya existentes. Acá podemos ver aplicada la lógica de diseño y creación de nuevo contenido en base a lo existente.

La vista presenta la información del evento y su imagen asociada tal y como se ve a continuación:

Venta de Tickets para Eventos

Tickets para Eventos

Detalle Evento: **Metallica**



#	Item	Valor
1	Nombre Evento	Metallica
2	Descripción	Descripción Uno
3	Precio	CL\$ 10000
4	Lugar	Estadio Nacional
5	Ver Mapa	Ver Mapa

Imagen 13. Vista detalle.jsp desde el navegador
Fuente: Desafío Latam

Vista JSP

Para esto, como siempre, el proceso es desde Eclipse, "File", "New", "JSP File"

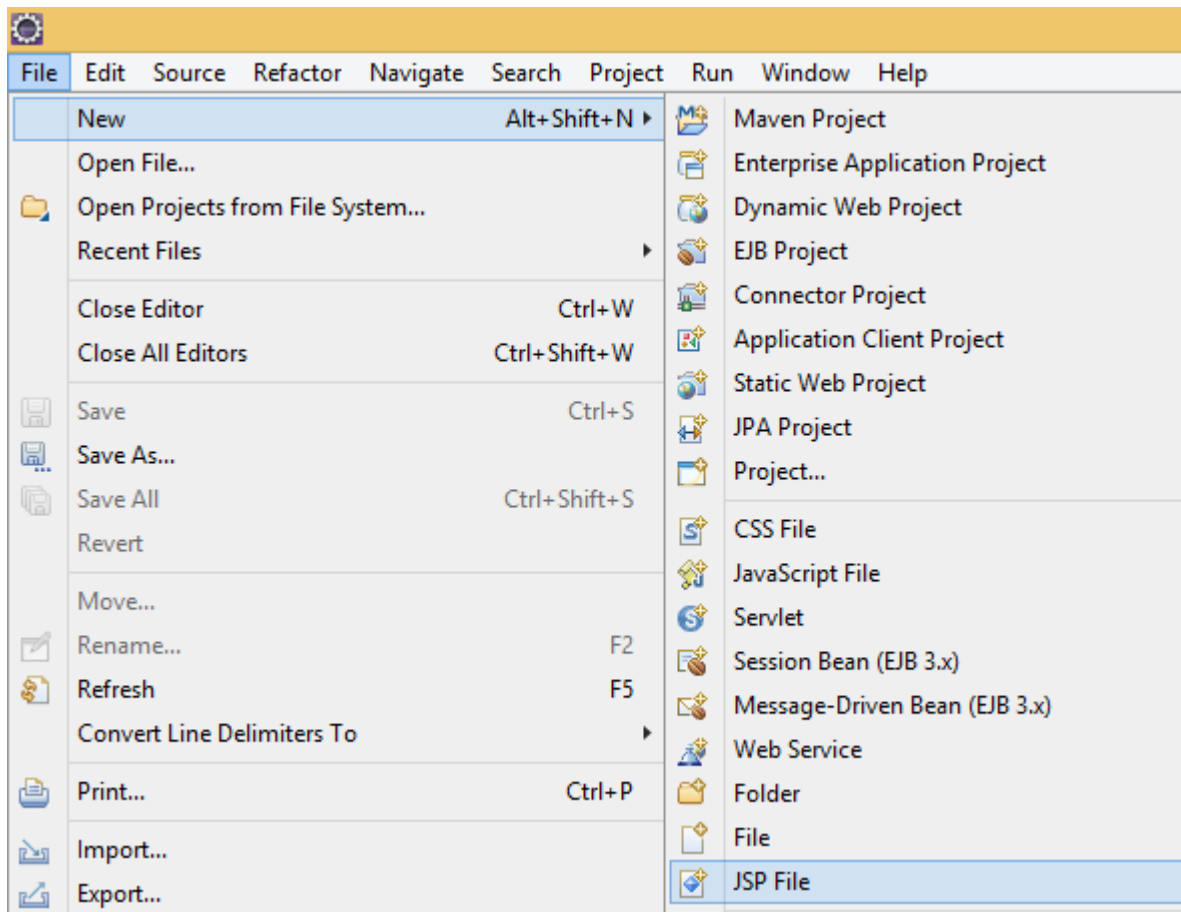


Imagen 14. Creación de una vista JSP
Fuente: Desafío Latam

detalle.jsp

Esta vista JSP toma desde request GET (vía URL) el parámetro de id evento, con el cual podemos buscar su detalle por medio de la función **"getEventobyID"**.

Para esto trabajamos con el objeto "Evento", el cual en principio se crea y establece el valor de id recogido por request, que finalmente es llenado al llamar a la función **"getEventobyID"**.

Recepcion valores desde request

```
int idEvento = Integer.parseInt(request.getParameter("idevento"));
```

Incluye Librerias requeridas

```
EventoBusiness eventob = new EventoBusiness();
```

Crea Objeto Evento y setea el id del mismo (el cual a su vez fue obtenido desde request)

```
Evento evento = new Evento();  
evento.setId(idEvento);
```

Busca el detalle del evento pasando el objeto respectivo

```
evento = eventob.getEventobyID(evento);
```

```
<%@page import="cl.desafiolatam.model.Evento"%>  
<%@page import="cl.desafiolatam.business.EventoBusiness"%>  
<%@ page language="java" contentType="text/html; charset=UTF-8"  
    pageEncoding="UTF-8"%>  
  
<%  
// Recepcion valores dede request  
int idEvento = Integer.parseInt(request.getParameter("idevento"));  
  
// Librerias requeridas  
EventoBusiness eventob = new EventoBusiness();  
  
// Crea Evento  
Evento evento = new Evento();  
evento.setId(idEvento);  
  
// Busca evento  
evento = eventob.getEventobyID(evento);  
%>  
<!DOCTYPE html>  
<html lang="es">  
<%@include file="assets/html/head.jsp"%>  
<body>  
  
    <%@include file="assets/html/header.jsp"%>  
  
    <!-- Section-->  
    <section class="py-5">  
        <div class="container px-4 px-lg-5 mt-5">  
  
            <div class="row justify-content-center">  
                <h1>  
                    Detalle Evento:  
<strong><%=evento.getNombre()%></strong>  
                </h1>  
                <hr/ >
```

```
        </div>
    </div>

    <div class="container px-4 px-lg-5 mt-5">

        " />

        <hr />

        <table class="table">
            <thead>
                <tr>
                    <th scope="col">#</th>
                    <th scope="col">Item</th>
                    <th scope="col">Valor</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <th scope="row">1</th>
                    <td>Nombre Evento</td>
                    <td><%=evento.getNombre()%></td>
                </tr>
                <tr>
                    <th scope="row">2</th>
                    <td>Descripción</td>
                    <td><%=evento.getDescripcion()%></td>
                </tr>
                <tr>
                    <th scope="row">3</th>
                    <td>Precio</td>
                    <td>CL$ <%=evento.getPrecio()%></td>
                </tr>
                <tr>
                    <th scope="row">4</th>
                    <td>Lugar</td>
                    <td><%=evento.getLugar()%></td>
                </tr>
                <tr>
                    <th scope="row">5</th>
                    <td>Fecha</td>
                    <td><%=evento.getFecha()%></td>
                </tr>
            </tbody>
        </table>
    </div>
```

```
                <th scope="row">5</th>
                <td>Ver Mapa</td>
                <td><a
href="mapa.jsp?idevento=<%=evento.getId()%>"
                class="btn btn-outline-dark
mt-auto">Ver Mapa</a></td>
            </tr>

        </tbody>
    </table>

</div>

</section>

<br /> <br />

<%@include file="assets/html/footer.jsp"%>

</body>
</html>
```

Salida de Proyecto

Finalmente, la salida del mismo queda del siguiente modo desde navegador

/actividad11/detalle.jsp?idevento=1 (id de evento seleccionado).

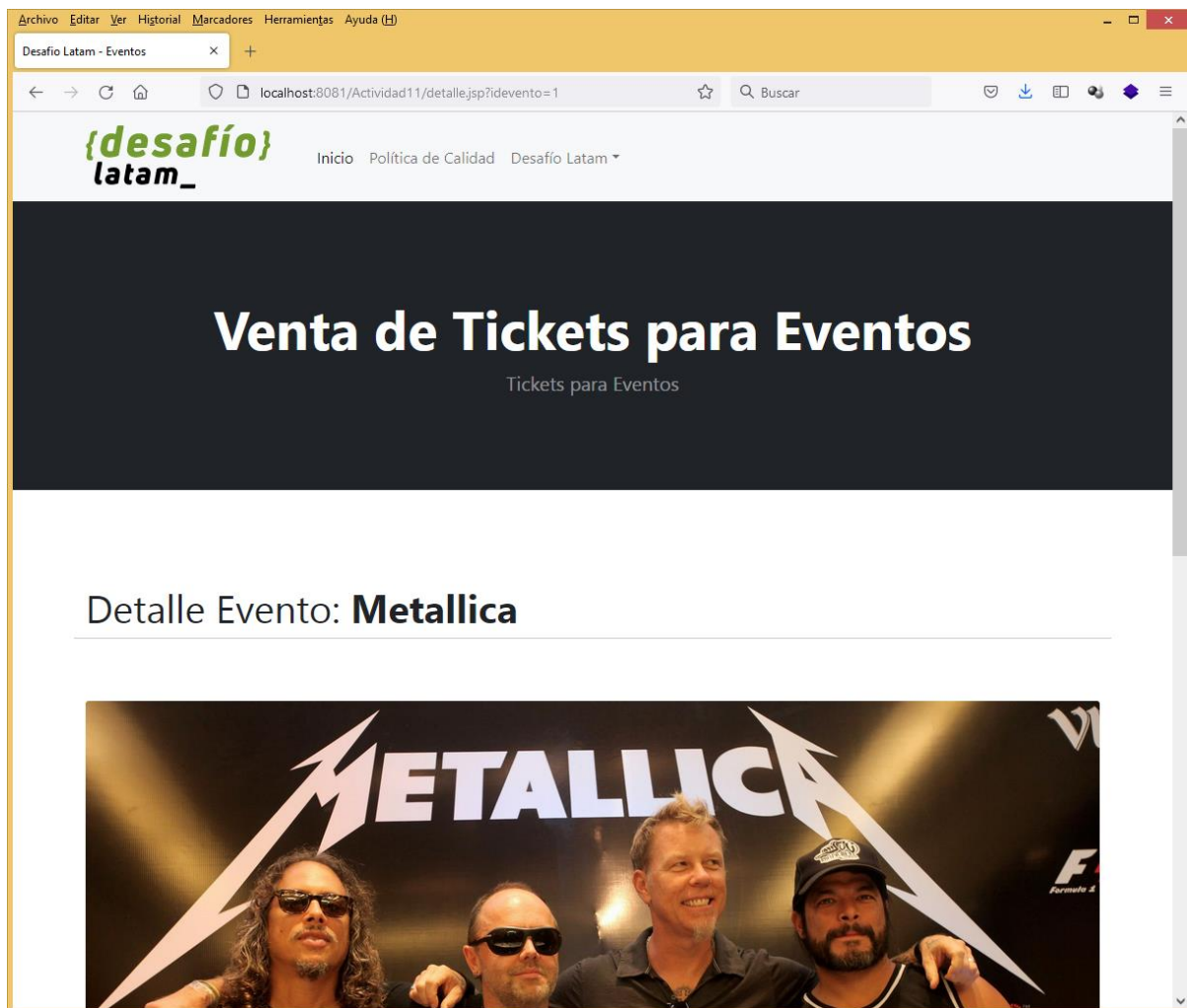


Imagen 15. Salida de Proyecto
Fuente: Desafío Latam



Actividad guiada - Agrega Mapa de Evento

Esta última actividad complementaria busca incorporar elementos anexos a nuestro proyecto.

En este sentido podemos observar que hay una gran cantidad de recursos de interés entregados por terceros. En particular, para este ejercicio tomaremos parte de los recursos que Google disponibiliza a los desarrolladores, y agregaremos el Mapa visual en donde el evento se realiza.

Para esto previamente recordemos que hemos creado nuestro objeto Evento, el cual tiene un atributo llamado "mapa". Este atributo, por tanto, posee el llamado de datos que luego le pasaremos vía parámetro a la URL de Google Maps, y que será incluida vía iframe a nuestro proyecto.

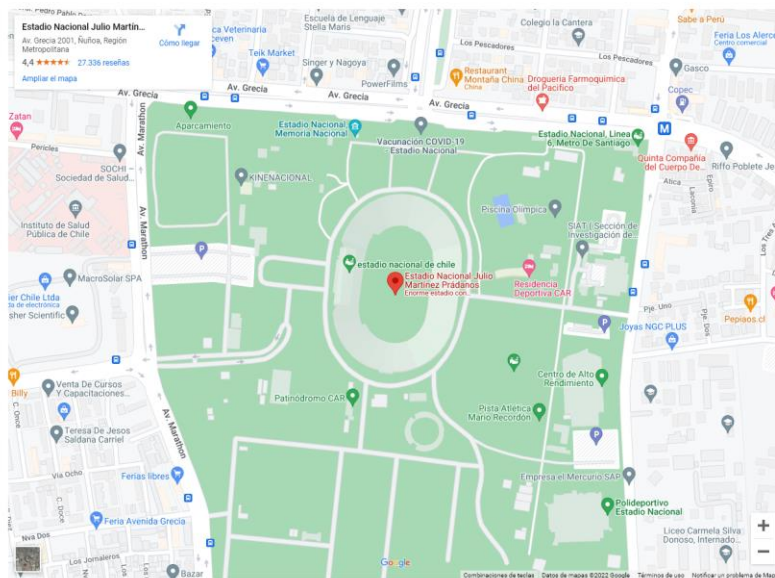
Para esto crearemos la página JSP en nuestro proyecto

mapa.jsp

Venta de Tickets para Eventos

Tickets para Eventos

Mapa Evento: **Metallica**



Desafío Latam - 2022

Imagen 16. Vista mapa.jsp desde el navegador
Fuente: Desafío Latam

Vista JSP

Como siempre, el proceso es desde Eclipse, **"File", "New", "JSP File"**

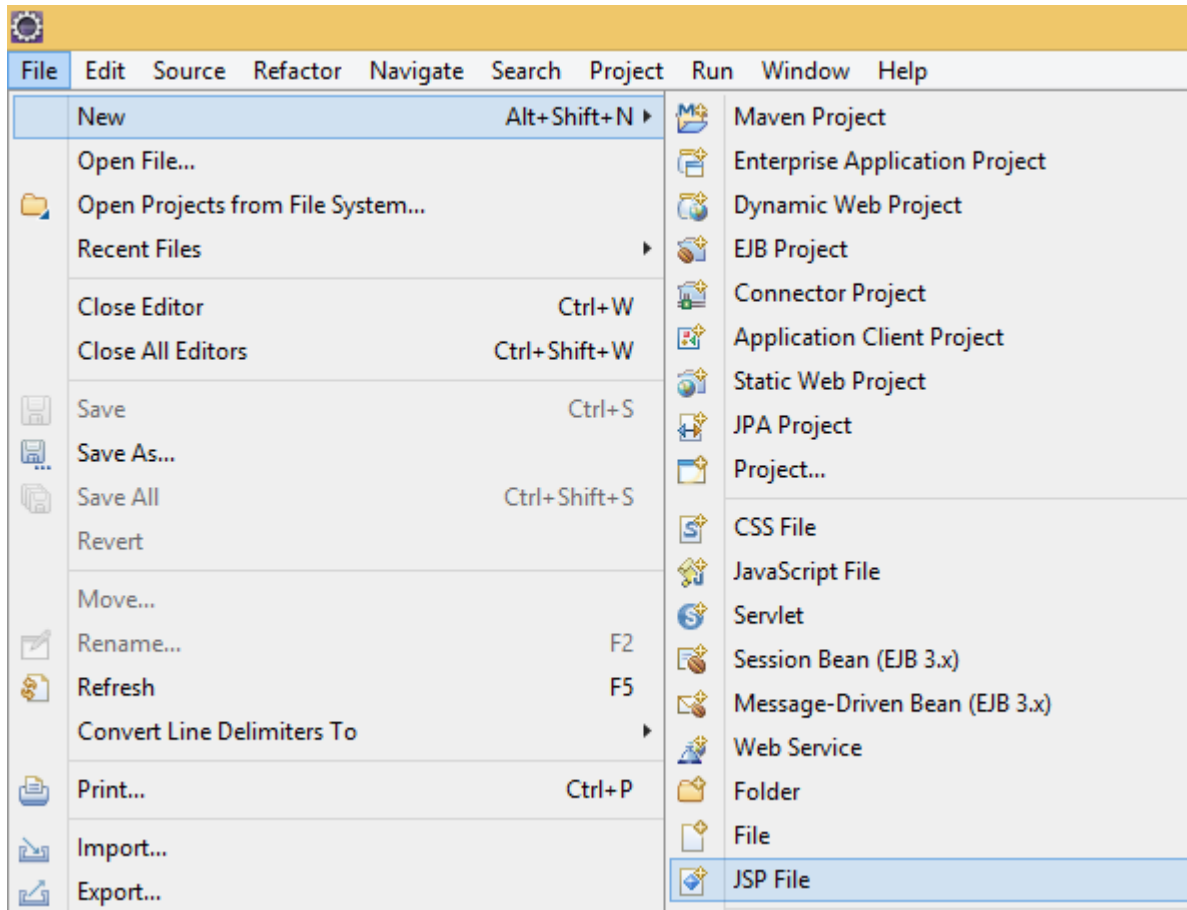


Imagen 17. Creación de un JSP
Fuente: Desafío Latam

mapa.jsp

El JSP mapa.jsp incluye la lógica para obtener el id de evento por medio de request GET (vía URL). Una vez capturado se crea el objeto "Evento" al cual le seteamos el id en el atributo respectivo.

Luego, buscamos su información por medio de la función **"getEventobyID"**, donde tendremos el objeto con todos los datos del evento, entre ellos la info para construir el mapa!

Recepcion valores desde request

```
int idEvento = Integer.parseInt(request.getParameter("idevento"));
```

Librerías requeridas

```
EventoBusiness eventob = new EventoBusiness();
```

Crea Evento

```
Evento evento = new Evento();  
evento.setId(idEvento);
```

Busca evento

```
evento = eventob.getEventobyID(evento);
```

Obtenida la información solo queda desplegarla en la página, para esto el proceso se reduce a crear el marco (iframe) respectivo con los datos del evento:

```
<iframe src="<%=evento.getMapa()%>" width="100%" height="900"  
style="border: 0;" allowfullscreen="" loading="lazy" referrerpolicy="no-  
referrer-when-downgrade"></iframe>
```

```
<%@page import="cl.desafiolatam.model.Evento"%>  
<%@page import="cl.desafiolatam.business.EventoBusiness"%>  
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"  
    pageEncoding="ISO-8859-1"%>  
<%  
//Recepcion valores dede request  
int idEvento = Integer.parseInt(request.getParameter("idevento"));  
  
//Librerias requeridas  
EventoBusiness eventob = new EventoBusiness();  
  
//Crea Evento  
Evento evento = new Evento();  
evento.setId(idEvento);  
  
//Busca evento  
evento = eventob.getEventobyID(evento);  
%>  
  
<!DOCTYPE html>  
<html lang="es">  
<%@include file="assets/html/head.jsp"%>  
<body>
```

```
<%@include file="assets/html/header.jsp"%>

<form method="post" action="procesa.jsp">

    <!-- Section-->
    <section class="py-5">
        <div class="container px-4 px-lg-5 mt-5">

            <div class="row justify-content-center">
                <h1>
                    Mapa Evento:
                </h1>
                <hr/ >
            </div>
        </div>

        <div class="container px-4 px-lg-5 mt-5">

            <iframe src="<%=evento.getMapa()%>" width="100%"
height="900"
                style="border: 0;" allowfullscreen=""
loading="lazy"
                referrerpolicy="no-referrer-when-
downgrade"></iframe>
        </div>

    </section>

    <br /> <br />

</form>

<%@include file="assets/html/footer.jsp"%>

</body>
</html>
```

Salida de Proyecto

Ya con esto podemos visualizar sin ningún problema el mapa respectivo al evento. Recordar que la información del mismo la hemos seteado en la clase Business de Evento.

```
// Info Evento 1
Evento evento1 = new Evento();
evento1.setId(1);
evento1.setNombre("Metallica");
evento1.setDescripcion("Descripción Uno");
evento1.setMapa("https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3328.411793967857!2d-70.61286488437952!3d-33.46462808076988!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x9662cfed639ab8133A0xb92488060556a1df!2sEstadio%20Nacional%20Julio%20Mart%C3%ADnez%20Pr%C3%A1danos!5e0!3m2!1ses!2sc1!4v1661265818941!5m2!1ses!2sc1");
```

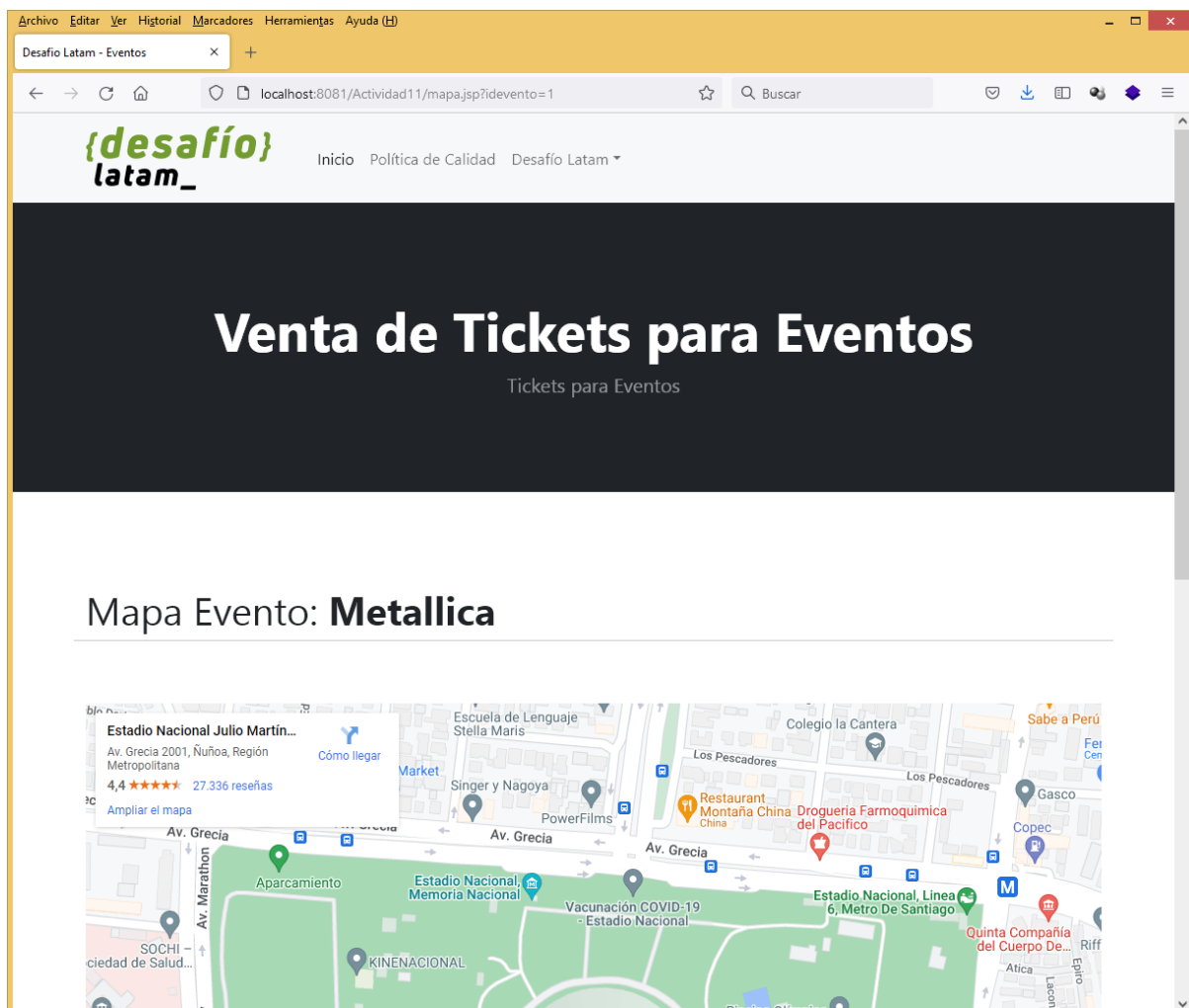


Imagen 18. Salida de Proyecto

Fuente: Desafío Latam

Preguntas de proceso

Reflexiona:

- ¿Qué aspectos nuevos he aprendido del lenguaje de programación Java?
- Respecto del ejercicio desarrollado, ¿hay algo que me haya resultado difícil de desarrollar o entender?
- ¿Qué te ha parecido trabajar con un esquema front con includes y uso de Bootstrap?



Preguntas de cierre

- Sobre formularios y envío de datos, ¿puedo distinguir la diferencia entre envío POST y GET?
- ¿Cuáles etiquetas de formulario me ayudan a establecer inputs de datos?
- ¿Cuál etiqueta HTML es la encargada de establecer la acción de envío de formulario?

Referencias bibliográficas

<https://getbootstrap.com/>

https://www.w3schools.com/html/html_forms.asp